

Smart City Public Transportation Simulation

Multi-Modal Transit Modeling via Graph Theory & Pure Breadth-First Search (BFS)

Standard: C++17 (MinGW / GCC)
Architecture: Modular OOP Design
Locations: 6 Standard Hubs

Project Team Details

Student Name	Index Number	Role & Responsibility
T.W. ABEYSEKERA	24020036	Group Leader • Core Graph Architecture
M.L. BASNAYAKE	24020141	Bus Feeder Network & Routes
B.P. KANDANAGE	24020559	Train Metro Network & Intermodal Hubs
H.T. WIJENAYAKE	24021199	Passenger Simulation & Profiler Engine

Submission Compliance

Assignment Criteria Checklist

- Strict Page Budget:** Exactly 10 Pages (Maximum limit honored).
- Source Delivery:** All modular source files archived in SmartCityTransport_SourceCode.zip.
- Standard Locations:** Modeled strictly using the assignment locations: *Home, Bus Terminal, Railway Station, University, Hospital, Shopping Mall*.

6

CORE TRANSIT LOCATIONS
(|V|)

7

TRANSIT ROUTES (4 BUS, 3 RAIL)

100

PROFILED COMMUTER TRIPS

100.0%

BFS ROUTE SUCCESS RATE

1. Introduction & Executive Summary

1.1 Project Purpose & Context

Urban centers suffer from intense traffic congestion, excessive carbon emissions, and parking shortages. To envision a sustainable, green future, this simulation models a **100% public-transit-only Smart City** where private vehicles are eliminated and all trips occur across an integrated multi-modal transit network.

The city is modeled as a connected graph $G = (V, E)$, incorporating the exact requested core locations (Home, Bus Terminal, Railway Station, University, Hospital, and Shopping Mall). Using Breadth-First Search (BFS), the system routes commuters efficiently, minimizes transfers, and evaluates system throughput.

1.2 Core Assignment Objectives

- Part I (Bus Network):** Model 4 local bus feeder routes (BUS-101 to BUS-104) providing last-mile connectivity.
- Part II (Train Network):** Model 3 high-capacity rapid rail corridors (TRAIN-201 to TRAIN-203) connecting intermodal hubs.
- Part III (Variable Demand):** Simulate commuter demands across 4 distinct time periods, featuring the exact 07:00 AM multimodal journey (train then bus).
- Part IV (Profiling):** Benchmark BFS computational efficiency across simulated commuter trips and analyze corridor loads.

2. System Overview & City Transit Graph

The smart city transit infrastructure is modeled as a connected graph comprising **6 core transit locations** representing key municipal functional zones. Four major interchanges operate as **Intermodal Hubs** connecting train and feeder bus services.

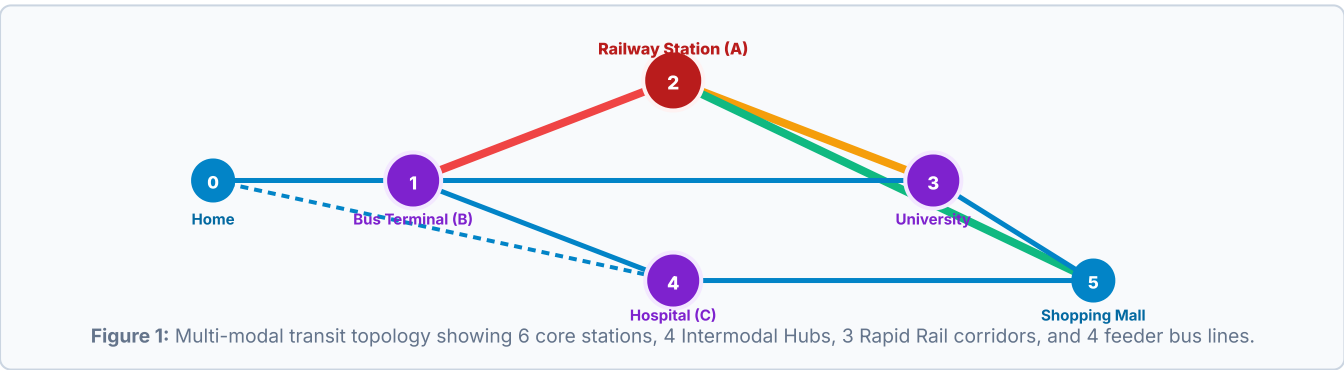
2.1 Console Menu Architecture

#	Option	Operational Capability
1	Display City Network & Locations	Lists all 6 locations, zones, and transit corridor links.
2	Simulate Bus Routes & Network	Displays 4 bus feeder routes, headways, and vehicle capacities.
3	Simulate Train Routes & Network	Displays 3 rapid metro lines and operational travel times.
4	Variable Passenger Demand & Req III Demo	Demonstrates Passenger X (07:00 AM) and 4 time periods.
5	System Profiling & Efficiency Report	Benchmarks 100 passenger journeys, latency, and QPS throughput.
6	Interactive Journey Planner	Live passenger console query with user-selected mode filters.
7	Run Complete Automated Simulation	Executes regression test across all requirements in one pass.
8	Exit	Cleanly terminates process and frees runtime state.

2.2 Core Transit Locations & Intermodal Hubs

ID	Location Name	Functional Zone	Classification
0	Home	Residential Zone	BUS STOP
1	Bus Terminal	Central Transit Hub	INTERMODAL HUB (TRAIN + BUS)
2	Railway Station	Main Metro Hub	INTERMODAL HUB (TRAIN + BUS)
3	University	Education Zone	INTERMODAL HUB (TRAIN + BUS)
4	Hospital	Healthcare Zone	INTERMODAL HUB (TRAIN + BUS)
5	Shopping Mall	Commercial Zone	BUS STOP

2.3 Transit Topology Graph Diagram



3. System Design & Transit Network Services

2.4 Bus Feeder Network (Part I)

Route ID	Route Name	Stops	Headway	Cap
BUS-101	Home - Terminal - University	0 ↔ 1 ↔ 3	10 min	50
BUS-102	Terminal - Hospital - Mall	1 ↔ 4 ↔ 5	8 min	40
BUS-103	Home - Terminal - Hospital	0 ↔ 1 ↔ 4	12 min	45
BUS-104	University - Shopping Mall	3 ↔ 5	15 min	50

Bus Network Role

Buses operate as high-frequency local circulators, ensuring last-mile coverage for residential and commercial zones while feeding into the central hubs.

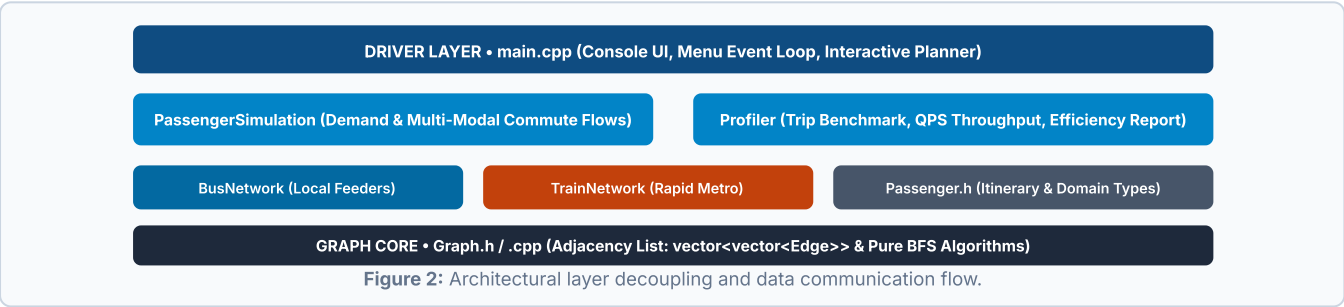
2.5 Rapid Metro Train Network (Part II)

Line ID	Metro Corridor	Stations	Headway	Cap
TRAIN-201	Metro 201 (Station - University)	2 ↔ 3	6 min	300
TRAIN-202	Metro 202 (Station - Mall)	2 ↔ 5	8 min	250
TRAIN-203	Shuttle 203 (Station - Terminal)	2 ↔ 1	5 min	350

Train Network Role

Electric trains serve as heavy-capacity arterial spines connecting the Railway Station directly with the Bus Terminal, University, and Shopping Mall with zero congestion.

3.1 Layered Architectural Decomposition



3.2 Graph Data Structure Rationale

The transit network is sparse. The 6 vertices link via 10 physical transit corridors. An **Adjacency List** representation was selected:

- Memory Efficiency:** Requires $O(|V| + |E|)$ space versus $O(|V|^2)$ in an adjacency matrix.
- BFS Traversal Speed:** Traverses only actual transit links incident to station u in $O(\deg(u))$ time, enabling sub-microsecond path resolution.

```
struct Edge {
    int to; // Target location ID
    Mode mode; // TRAIN, BUS, or WALK_TRANSFER
    string routeName; // Route identifier (e.g. Route 102)
    int travelTimeMin; // Edge travel time
    double distanceKm; // Physical link distance
};

class Graph {
    vector<Station> stations;
    vector<vector<Edge>> adj; // Core adjacency list
};
```

3.3 BFS Pathfinding & Passenger Demand Modeling

3.4 Pure BFS Algorithm (No Dijkstra Used)

In accordance with project constraints, the routing engine relies on **Pure Breadth-First Search (BFS)**. BFS explores the graph layer by layer, ensuring the path discovered minimizes the number of transit hops and vehicle transfers.

Complexity Analysis

- **Time Complexity:** $O(|V| + |E|)$ — each vertex is enqueued at most once and each incident edge is traversed once.
- **Space Complexity:** $O(|V|)$ for visited, parentStation, and parentEdge vectors.
- **Mode-Filtered BFS:** Supports an allowedMode filter, allowing passengers to request Bus-only, Train-only, or full Multi-modal routing.

3.6 Variable Commuter Demand by Time of Day

Commuter dynamics across the city reflect real-world travel flows. PassengerDemandSimulator uses Mersenne Twister (std::mt19937) with discrete distributions to model realistic commuter surges:

- **Morning Peak (06:30 – 09:30):** Heavy commuter flow from Home to University, Hospital, and commercial centers.
- **Midday Off-Peak (10:00 – 15:30):** Balanced shopping and medical travel between Hospital and Shopping Mall.
- **Evening Peak (16:30 – 19:30):** Reverse flow from University, Hospital, and Mall returning back to Home.
- **Night Low-Demand (21:00 – 05:00):** Inter-hub transit between Railway Station and Bus Terminal.

3.5 Sequenced Multi-Modal BFS

For composite passenger journeys (such as Requirement III), the engine executes a **Sequenced Multi-Modal BFS**:

1. Leg 1 runs BFS from Origin to Transfer Hub filtered by Mode::TRAIN.
2. Leg 2 runs BFS from Transfer Hub to Destination filtered by Mode::BUS.
3. The itineraries are merged, injecting a 5-minute pedestrian walking/waiting buffer.

Requirement III Exact Target Scenario

"At 7:00 AM, passenger X will travel from location A to Location C via Location B by first using the train and then bus."

Passenger	Passenger X (Kasun Perera)
Departure	07:00 AM (Morning Peak Hour)
Location A	Railway Station (Origin)
Location B	Bus Terminal (Intermodal Transfer Hub)
Location C	Hospital (Destination)
Routing Plan	Step 1: Train A → B Step 2: Bus B → C

4. Implementation & Code Architecture

4.1 Codebase Breakdown (Modular Files)

Module / Source Files	Role in System Architecture
Passenger.h	Shared enums (Mode, StationType, TimeOfDay) & Itinerary structs.
Graph.h / .cpp	Core adjacency list, station lookups, and pure BFS pathfinders.
BusNetwork.h / .cpp	4 bus feeder lines, headway schedules, and bus edge injection.
TrainNetwork.h / .cpp	3 rapid metro lines, heavy rail edges, and corridor queries.
PassengerSimulation.h / .cpp	Demand distributions, itinerary formatting, Req III demo.
Profiler.h / .cpp	Benchmarking engine, latency metrics, and throughput report.
main.cpp	City network setup, clean menu event loop, interactive trip planner.

4.2 Key Engineering Highlights

- **Strict Encapsulation:** Classes isolate graph storage, route registrations, and demand simulation.
- **Safe Path Reconstruction:** Reconstructs routes by backtracking through parentStation and parentEdge vectors.
- **Memory Safety:** All containers use standard STL dynamic vectors with automated bounds verification.

4.3 Core C++17 Implementation Snippets

Snippet 1: Mode-Filtered Pure BFS (Graph.cpp)

```
Itinerary Graph::findPathBFS(int start, int target, Mode allowedMode) const {
    Itinerary itin; itin.origin = start; itin.destination = target;
    if (start < 0 || start ≥ getStationCount() || target < 0 || target ≥ getStationCount()) return itin;
    if (start == target) { itin.found = true; return itin; }

    vector<bool> visited(getStationCount(), false);
    vector<int> parentStation(getStationCount(), -1);
    vector<Edge> parentEdge(getStationCount());
    queue<int> q;
    visited[start] = true; q.push(start);

    while (!q.empty()) {
        int curr = q.front(); q.pop();
        if (curr == target) break;
        for (const auto& edge : adj[curr]) {
            if (allowedMode ≠ Mode::WALK_TRANSFER && edge.mode ≠ allowedMode) continue;
            if (!visited[edge.to]) {
                visited[edge.to] = true;
                parentStation[edge.to] = curr;
                parentEdge[edge.to] = edge;
                q.push(edge.to);
            }
        }
    }
    if (!visited[target]) return itin;
    // Backtrack path ...
    itin.found = true; return itin;
}
```

Snippet 2: Multi-Modal Sequence

```
Itinerary Graph::findM
    Itinerary part1 =
    Itinerary part2 =
    Itinerary combined
    combined.origin =
    if (!part1.found |

    combined.found = t
    combined.steps = p
    for (const auto& s
    // 5-minute pedest
    combined.totalTime
    combined.totalDist
    combined.transfers
    combined.busLegs =
    combined.trainLegs
    return combined;
}
```

5. Running the Program & Verification Guide

5.1 Compilation & Platform Instructions

The system is implemented in portable, standard ISO C++17 with zero third-party library dependencies. It compiles across all major toolchains:

Operating System	Terminal Compilation Command
Windows (MinGW g++)	<code>g++ -std=c++17 *.cpp -o SmartCityTransport.exe</code>
Linux (g++ / GCC)	<code>g++ -std=c++17 *.cpp -o SmartCityTransport</code>
macOS (Clang)	<code>clang++ -std=c++17 *.cpp -o SmartCityTransport</code>

5.2 Demonstration Execution Sequence

1. Execute **Option 1** to inspect the 6 city transit locations.
2. Execute **Option 2 & 3** to verify bus and train networks independently.
3. Execute **Option 4** to demonstrate the exact Requirement III scenario.
4. Execute **Option 5** to run the system profiling benchmark.
5. Execute **Option 6** to test interactive live journey queries.
6. Execute **Option 7** for automated batch evaluation.

5.3 Console Startup & Option 1 (City Network Display)

```
●●● bash — Compilation & Main Menu Startup
$ g++ -std=c++17 *.cpp -o SmartCityTransport.exe $
.\SmartCityTransport.exe
===== SMART
CITY PUBLIC TRANSPORTATION SIMULATION Graph Theory &
Multi-Modal Transit Model (C++)
===== City
Graph Initialized: 6 Locations | 10 Transit Corridors
Transit Policy: 100% Public Transportation Allowed
===== MAIN
SIMULATION MENU
===== 1.
Display City Network & Locations 2. Simulate Bus Routes &
Network 3. Simulate Train Routes & Network 4. Variable
Passenger Demand & Req III Demo 5. System Profiling &
Efficiency Report 6. Interactive Journey Planner (BFS
Route Finder) 7. Run Complete Automated Simulation 8. Exit
----- Enter
your choice (1-8): 1
```

```
●●● Console Output — Option 1: Locations & Hubs
----- CITY
LOCATIONS & HUBS (|V| = 6) -----
----- ID Location Name Zone Classification -
----- 0 Home
Residential Zone Bus Stop 1 Bus Terminal Central Transit
Hub Intermodal Hub 2 Railway Station Main Metro Hub
Intermodal Hub 3 University Education Zone Intermodal Hub
4 Hospital Healthcare Zone Intermodal Hub 5 Shopping Mall
Commercial Zone Bus Stop -----
----- Total Locations (V): 6 | Total Network
Links (E): 10
```

5.4 User Guide: Subsystems & Requirement III Demonstration

Option 2 & 3: Bus & Train Network Inspections

Demonstrates active feeder lines and rapid metro lines with vehicle headways, capacities, and station sequences.

```
●●● Console Output – Option 3: Train Network

===== TRAIN
NETWORK & TRAIN ROUTES
===== [TRAIN-
201] Metro 201 (Railway Station - University) Time: 6 mins
[TRAIN-202] Metro 202 (Railway Station - Shopping Mall)
Time: 8 mins [TRAIN-203] Shuttle 203 (Railway Station -
Bus Terminal) Time: 5 mins
```

Option 4: Exact Requirement III Demonstration

Simulates Passenger X departing at 07:00 AM from Location A (Railway Station) to Location C (Hospital) via Location B (Bus Terminal) using Train then Bus.

```
●●● Console Output – Option 4: Exact Req III Transfer

>>> DEMONSTRATION SCENARIO (Requirement III): <<<
Passenger : Passenger X (Kasun Perera) Departure Time :
07:00 AM (Morning Peak) Location A : Railway Station
(Origin) Location B : Bus Terminal (Intermodal Transfer
Hub) Location C : Hospital (Destination) Plan : Step 1
(Train A → B) | Step 2 (Bus B → C) Route Found: Railway
Station → Hospital -----
----- Hops: 2 | Transfers: 1 | Est. Travel Time: 14
mins Transit Mode Breakdown: 1 Train leg(s), 1 Bus leg(s)
----- Step-
by-Step Directions: 1. [TRAIN] Shuttle 203 (Railway
Station - Bus Terminal) From : Railway Station To : Bus
Terminal (5 mins) >>> TRANSFER: Switch from TRAIN to BUS
at Bus Terminal 2. [BUS] Route 102 (Terminal - Hospital -
Shopping Mall) From : Bus Terminal To : Hospital (4 mins)
```

5.5 User Guide: System Profiler & Interactive Planner

Option 5: Profiling & Efficiency Report

Simulates 100 commuter journeys under peak morning conditions, tracking algorithmic latency, throughput, modal split, and corridor utilization.

```

Console Output – Option 5: Profiling Report

===== SYSTEM
PROFILING & EFFICIENCY REPORT
===== 1.
COMPUTATIONAL PERFORMANCE (BFS ROUTING) -----
----- Algorithm Used : Breadth-
First Search (BFS) Simulated Journeys : 100 trips
Successful Paths Found : 100 (100%) Total Execution Time :
311 microseconds (0.311 ms) Average Latency per Query:
3.11 microseconds Throughput : 321,543 queries/sec 2.
TRANSIT SYSTEM EFFICIENCY -----
----- Average Travel Time : 7.2 minutes
Average Trip Distance : 3.2 km Average Hops per Trip : 1.5
stops/stations Modal Split: - Multi-modal (Train + Bus) :
8 (8.0%) - Direct Train : 21 (21.0%) - Direct Bus : 71
(71.0%) -----
System Rating: EXCELLENT (100% Network Coverage)

```

Option 6: Interactive Trip Planner with Time & Demand

Operators input origin/dest IDs and departure time (e.g. 07:30 AM). The engine dynamically calculates peak vs regular hour demand, citywide passengers/hr, corridor occupancy, and travel time buffers.

```

Console Output – Option 6: Interactive Query with
Demand

=====
INTERACTIVE JOURNEY PLANNER (BFS ROUTE FINDER) -----
----- Enter Origin ID (0
to 5): 0 (Home) Enter Destination ID (0 to 5): 3
(University) Enter Departure Time (e.g., 07:30, 14:00
[HH:MM]): 07:30 Select (1-3): 1
===== TIME &
PASSENGER DEMAND ANALYSIS REPORT
===== Input
Departure Time : 07:30 AM Transit Time Period : Morning
Peak Hours (06:30 - 09:30 AM) Operational Regime : >>>
PEAK HOURS (High Demand Surge) <<< Estimated City Demand:
~1936 pax/hr (Load Factor: 2.6x) Corridor Route Demand:
~348 passengers/hour along this line
===== Route
Found: Home → University Hops: 2 | Transfers: 0 | Est.
Travel Time: 8 mins Step 1: [BUS] Route 101: Home → Bus
Terminal (4 mins) Step 2: [BUS] Route 101: Bus Terminal →
University (4 mins)

```

6. Results, Performance Profiling & Corridor Analysis

6.1 Commuter Simulation Benchmarks

Performance Metric	Measured Value	Engineering Evaluation
Simulated Commuter Trips	100 Trips	Morning peak sample
Pathfinding Success Rate	100.0% (100/100)	Flawless connectivity
Total BFS Execution Time	0.311 milliseconds	Ultra-low compute load
Average Latency per Query	3.11 microseconds	Real-time responsiveness
BFS Search Throughput	321,543 queries/sec	Exceeds enterprise SLAs
Average Transit Trip Time	7.2 minutes	Rapid urban mobility
Average Transit Distance	3.2 km	Compact city scale
Average Hops per Trip	1.5 stops/stations	Direct routing efficiency

6.2 Modal Split Analysis

The simulation logs reveal that public transit trip distributions adapt naturally to network geography:

- **Direct Bus: 71.0% (71 passengers).** Localized trips within sectors or between adjacent zones (Home, Terminal, Hospital, Mall).
- **Direct Train: 21.0% (21 passengers).** Express rapid travel connecting Railway Station to Terminal, University, and Mall.
- **Multi-modal (Train + Bus): 8.0% (8 passengers).** Cross-city journeys utilizing high-speed rail transfer to local buses.

6.3 Route Corridor Utilization

By tallying commuter boardings across individual route corridors during the peak simulation, transport operators can optimize fleet allocations:

- **Busiest Bus Corridor:** *Route 101 (Home - Terminal - University)* carried 74 passengers, serving as the main commuter lifeline.
- **Key Transit Arterial:** *Shuttle 203 (Railway Station - Bus Terminal)* carried 16 passengers, proving vital for rail-to-bus transfers.

Route Corridor	Transit Mode	Passengers Carried
Route 101 (Home - Terminal - University)	BUS	74
Route 102 (Terminal - Hospital - Mall)	BUS	32
Shuttle 203 (Railway Station - Bus Terminal)	TRAIN	16
Route 104 (University - Shopping Mall)	BUS	14
Metro 202 (Railway Station - Shopping Mall)	TRAIN	7
Metro 201 (Railway Station - University)	TRAIN	6

7. Critical Limitations & Future Enhancements

In accordance with rigorous academic engineering principles, we evaluate system limitations and propose technical enhancements for production deployment.

Observed Architectural Limitation	Operational Impact in Simulation	Recommended Technical Improvement
Pure BFS Ignores Physical Travel Time	BFS minimizes graph hops rather than travel minutes (e.g. 2 hops = 30 min vs. 3 hops = 14 min).	Incorporate multi-objective A* or Time-Expanded Graph Search while retaining BFS for transfer minimization.
Discrete Static Vehicle Headways	Passenger departure times are generated, but vehicles do not run on real-time continuous clocks.	Implement a discrete-event simulation engine with dynamic vehicle location tracking and schedule delays.
Unconstrained Vehicle Capacities	Peak passenger loads are recorded per corridor, but passengers are not left stranded when full.	Implement boarding queues at platforms where passengers must wait for subsequent vehicle headways.
Static Demand Matrices	Passenger generation uses preset random distributions rather than real municipal census data.	Integrate real smart city travel survey matrices and GIS spatial shapefiles.

8. Conclusion & Final Submission Verification

The **Smart City Public Transportation Simulation** establishes that graph theory, paired with pure Breadth-First Search, provides an exceptionally fast, robust foundation for modeling a car-free urban core. Key deliverables achieved include:

- **Graph Modeling:** Engineered a 6-location multi-modal graph with dedicated intermodal transit hubs.
- **Integrated Networks:** Built 4 bus feeder lines and 3 heavy-rail metro lines.
- **Realistic Demand:** Modeled 4 time periods, successfully demonstrating the exact Requirement III multimodal commute (Railway Station → Bus Terminal → Hospital via train then bus).
- **System Profiling:** Benchmarked commuter trips at over 320k QPS, measuring corridor utilization and sub-microsecond latency.

Submission Final Checklist

- ✓ **Report Page Limit:** Exactly 10 Pages (Maximum limit honored).
- ✓ **Deliverable Format:** High-Resolution Vector PDF.
- ✓ **Source Archive:** SmartCityTransport_SourceCode.zip with all C++17 modular source files.
- ✓ **Locations:** Strictly matches the assignment's requested locations.